Generarea cercurilor în spatiul discret

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Aproximarea cercurilor in spatiul discret(1)

- Cercurile si elipsele nu sunt primitive grafice in OpenGL, deci nu pot fi transmise in banda grafica.
- Cercurile si elipsele sunt primitive 2D definite in planul XOY.
- Pentru afisare ele sunt descompuse in primitive de tip vector sau primitive de tip punct:
  - le putem asocia coordonata z si transmite in banda grafica sub forma unei secvente de vectori sau puncte 3D
  - mai eficient: aproximarea cercurilor prin vectori sau puncte se poate face in “Geometry shader”
- Fie (xc,yc,0) coordonatele centrului unui cerc/elipsa. Pentru afisarea sa vom folosi functiile:

  point (int x, int y, RGB culoare);

  - functia genereaza o primitiva “point” avand coordonatele (x,y,0) si culoarea specificata.

  line (int x1, int y1, int x2, int y2, RGB culoare)

  - functia genereaza o primitiva “line” cu varfurile (x1,y1, 0) - (x2,y2,0) si culoarea specificata asociata ambelor varfuri.

Aproximarea cercurilor in spatiul discret(1)

1) Folosind ecuatia cercului in coordonate carteziene

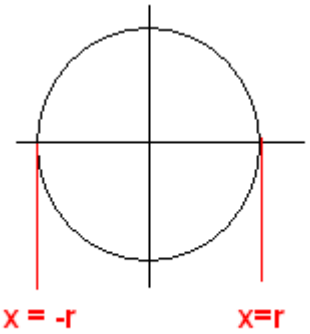
$(x - xc)^2 + (y - yc)^2 = r^2$

Generarea cercului cu centrul in (xc,yc) se reduce la generarea cercului centrat in origine, plus o translatie cu (xc, yc) a coordonatelor.

$x^2 + y^2 = r^2$

Se incrementeaza x de la –r la r si se calculeaza y din ecuatia cercului:

```
point (xc - r, yc, culoare);
point (xc + r, yc, culoare);
for (x = -r +1; x< r; x++)
{   y = sqrt(r^2 – x^2);
    point (xc + x, yc + y, culoare);
    point (xc + x, yc - y, culoare);
}
```

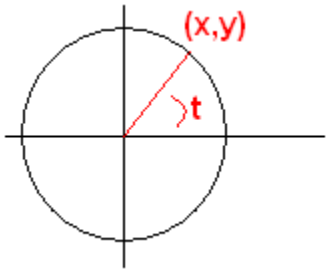


Dezavantaje:

- Spatierea inegala a punctelor de pe cerc afişate
- Calcule complexe cu numere reale

Aproximarea cercurilor in spatiul discret(2)

2) Folosind ecuatia cercului in coordonate polare



$x = xc + r^* \cos(t)$

$y = yc + r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Se genereaza cercul centrat in origine si se aplica punctelor rezultate o translatie cu (xc, yc):

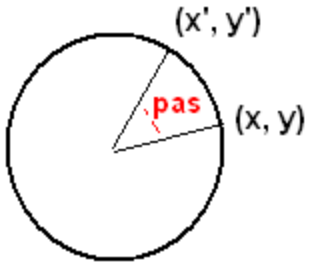
$x = r^* \cos(t)$

$y = r^* \sin t(t)$ $\quad 0 <= t <= 2\pi$

Cercul poate fi aproximat pintr-o polilinie:

- daca se calculeaza N puncte de pe cerc, egal distantate, t creste cu pasul $2\pi/N$

Calculul punctelor succesive de pe cerc, pe baza unei relatii de recurenta:



- (x, y): punctul curent

- (x', y'): punctul urmator

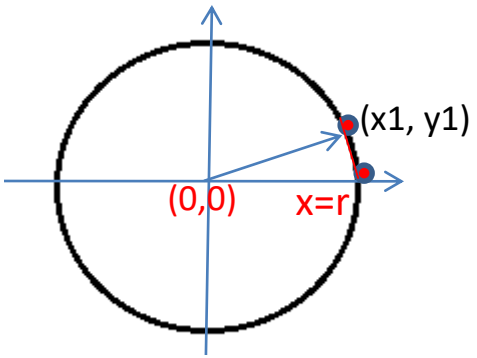
$x' = r^* \cos(t + pas) = r(\cos(t) * \cos(pas) - \sin(t)* \sin(pas)) = x*\cos(pas) - y*\sin(pas)$

$y' = r^* \sin(t + pas) = r(\cos(t) * \sin(pas) + \sin(t)* \cos(pas)) = x*\sin(pas) + y*\cos(pas)$

Aproximarea cercurilor in spatiul discret(3)

Avantajul calculului prin recurenta: nu este necesar sa se calculeze sin si cos in fiecare iteratie!

```
// Aproximarea unui cerc printr-o linie poligonala
void Cerc( int xc, int yc, int r, int N, RGB culoare)
{ double pas = 2*M_PI /N ; //pasul unghiular
  double u, x, y, x1,y1, s, c;
  c=cos(pas); s=sin(pas); // sin si cos se calculeaza o singura data
  for( int  i = 1, x=r, y=0; i<N; i++)
  { x1 =x*c - y*s; // x'
    y1 =x*s + y*c;  // y'
    line(xc+(int)(x+0.5), yc+(int)(y+0.5), xc+(int)(x1+0.5), yc+(int)(y1+0.5), culoare);
    x=x1 ; y= y1 ;
}
line(xc+(int)(x+0.5), yc+(int)(y+0.5),xc+r,yc, culoare);
}
```



Aproximarea cercurilor in spatiul discret(4)

Simetria punctelor de pe cerc

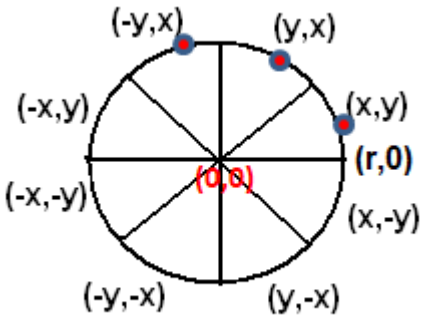
- **Fiecare punct de pe cerc are 7 puncte simetrice pe cerc:**

  - Este suficient sa se calculeze adresele punctelor dintr-un singur octant al cercului, celelalte puncte de pe cerc find calculate prin simetrie

// Afiseaza punctele simetrice de pe cerc

void punct_simetric(int xc,int yc,int x,int y, RGB culoare)

```
{ point(xc+x,yc+y,culoare);
  point(xc+x,yc-y,culoare);
  point(xc-x,yc-y,culoare);
  point(xc-x,yc+y,culoare);
  point(xc+y,yc+x,culoare);
  point(xc+y,yc-x,culoare);
  point(xc-y,yc-x,culoare);
  point(xc-y,yc+x,culoare);
}
```



Obs:

Simetricele punctului (r,0): (r, 0), (-r, 0), (-r,0), (0,r), (0,-r), (0,-r), (0,r)

→ pixelii (xc+r,yc), (xc-r,yc), (xc,yc+r), (xc,yc-r) vor fi suprascrisi

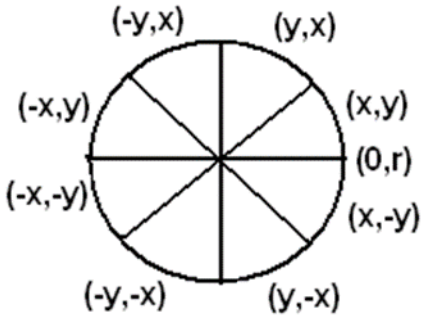
→ nu va fi apelata functia pentru punctul (r,0)

Aproximarea cercurilor in spatiul discret(5)

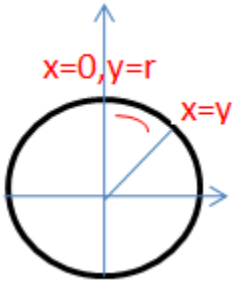
- Pentru afisarea punctelor succesive de pe circumferintă, pasul unghiular trebuie sa fie aprox. (1/ raza).

// Calculeaza numai punctele din primul octant si afiseaza toate punctele simetrice

```
void Cerc_sim( int xc, int yc, int r, RGB culoare)
{ double pas, u, x, y, xx, s, c;
  pas=1.0/r; // pas unghiular pentru obtinerea de puncte succesive de pe cerc
  c=cos(pas); s=sin(pas);
  point(xc+r,yc,culoare);
  point(xc-r,yc,culoare);
  point(xc,yc+r,culoare);
  point(xc,yc-r,culoare);
  for(u=pas, x=r, y=0; u<M_PI_4; u+=pas)
  { xx=xx;
    x =xx*c - y*s;
    y =xx*s + y*c;
    punct_simetric(xc, yc,(int)(x+0.5), (int)(y+0.5), culoare);
  }
}
```

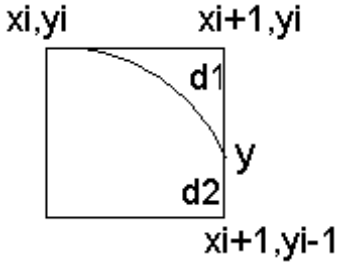


Algoritmul Bresenham pentru aproximarea cercurilor în spatial discret(1)



- Consideram cercul cu centrul în originea sistemului de coordonate si raza r.
- In cadrul algoritmului **se calculeaza numai punctele din octantul al 2-lea,** celelalte obtinandu-se prin simetrie.
- Se calculeaza punctele de pe cerc incepand cu (x=0,y=r) pana la (x=y), prin incrementarea lui x.
- Fie (x_i,y_i) ultimul punct al spatiului discret, ales pentru aproximarea cercului.

  Urmatorul punct va fi unul dintre (x_i+1,y_i) si (x_i+1,y_i-1):



Fie y ordonata punctului de pe cercul teoretic, cu abscisa x_i+1.

$$y^2 = r^2-(x_i+1)^2$$

Fie d1 si d2 distantele de la cele 2 puncte ale spatiului discret la punctul de pe cercul teoretic.

Intereseaza semnul diferentei (d1-d2):

- daca $(y_i - y) > (y- (y_i-1))$ atunci

$$(y_i^2 - y^2) > (y^2 - (y_i-1)^2)$$

$$d1' = y_i^2 - y^2 = y_i^2 - r^2 + (x_i+1)^2$$

$$d2' = y^2 - (y_i-1)^2 = r^2 - (x_i+1)^2 - (y_i-1)^2$$

Algoritmul Bresenham (2)

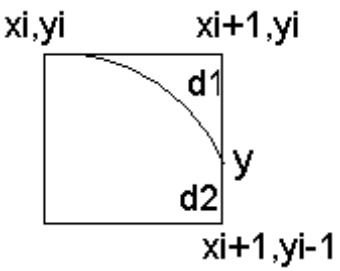
- Notam cu t_i eroarea de aproximare în pasul curent:

$t_i = d1'-d2' = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 -2*r^2$

- Se obtine o relatie de recurenţă pentru calculul erorii de aproximare in pasul urmator:

$t_{i+1}= y_{i+1}^2 + 2*(x_{i+1}+1)^2 + (y_{i+1}-1)^2 -2*r^2$

(1) Daca **$t_i<0$** ($d1' < d2'$) atunci $x_{i+1}=x_i+1$ si $y_{i+1}=y_i$. Deci,



$t_{i+1}= y_i^2 + 2*((x_i+1)+1)^2 + (y_i-1)^2 -2*r^2$

sau **$t_{i+1} = t_i + 4*x_i + 6$**

(2) Daca **$t_i >=0$** atunci $x_{i+1} = x_i + 1$ si $y_{i+1} = y_i - 1$. Deci,

$t_{i+1} = (y_i-1)^2 + 2*((x_i+1)+1)^2 + ((y_i-1)-1)^2 -2*r^2$

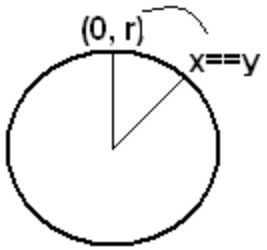
sau **$t_{i+1} = t_i + 4*(x_i - y_i) + 10$**

Algoritmul Bresenham (3)

- Valoarea erorii de aproximare în primul pas:

$t_i = y_i^2 + 2*(x_i+1)^2 + (y_i-1)^2 - 2*r^2$: $x_1=0$, $y_1 = r$. Rezultă, **t1 = 3-2*r**

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  point(xc+r,yc,culoare); point(xc-r,yc,culoare);
  point(xc,yc+r,culoare); point(xc,yc-r,culoare);
  for(x=1,y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
    punct_simetric(xc,yc,x,y,culoare);
  }
// pentru x==y se afiseaza 4 puncte simetrice
  point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
  point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```



Generarea unei suprafete circulare

EXERCITIU:

- Modificati algoritmul Bresenham pentru generarea cercurilor astfel incat sa se obtina o suprafata circulara.

```
void Bres_cerc(int xc, int yc, int r, RGB culoare)
{ int x,y,t;
  t=3-(r<<1);
  line(xc, yc+r, xc, yc-r, culoare); point(xc-r, yc, culoare); point(xc+r, yc, culoare);
  for(x=1, y=r; x<y; x++)
  {
    if(t<0)
      t+=6+(x<<2);
    else
     { t+=10+((x-y)<<2); y--;}
   line_simetric(xc,yc,x,y,culoare);//traseaza 4 linii simetrice
  }
 point(xc+x,yc+y,culoare); point(xc+x,yc-y,culoare);
 point(xc-x,yc+y,culoare); point(xc-x,yc-y,culoare);
}
```

De ce linii verticale si nu orizontale?

